

Universidade Tecnológica Federal do Paraná – Campus Apucarana

Comunicação de Dados – Trabalho – 1º semestre

“Trabalho sobre segunda parte do conteúdo”

Prof. Dr. Luiz Fernando Carvalho

Integrantes: Elder Nunes Gonçalves | Felipe Dos Santos Rodrigues | Daniel Kroda

Exercício 1

[1,0 ponto]

a) Portadoras**Sim, existem duas portadoras.**

Analisando a Figura 1, observam-se dois espectros de banda passante centrados em regiões distintas do espectro audível da linha telefônica (300–3400 Hz). A frequência fundamental de cada portadora é o ponto central de cada espectro:

- **Portadora 1:** centrada em $f_{c1} = \frac{1070 + 1270}{2} = \mathbf{1170\ Hz}$

- **Portadora 2:** centrada em $f_{c2} = \frac{2025 + 2225}{2} = \mathbf{2125\ Hz}$

Cada portadora é modulada para representar sinais em uma direção de transmissão diferente, dentro da faixa utilizável da linha telefônica.

b) Modo de transmissão

O modo de transmissão é **full-duplex**.

A figura mostra dois espectros distintos utilizados *simultaneamente*: um para transmissão em uma direção (1070–1270 Hz) e outro para a direção oposta (2025–2225 Hz). Como ambas as direções operam ao mesmo tempo, sem necessidade de alternância, caracteriza-se o modo full-duplex. Isso é possível pela separação em frequência (FDM), que permite que os dois fluxos coexistam no mesmo meio físico sem interferência.

c) Multiplexação

Sim, ocorre multiplexação – especificamente **Multiplexação por Divisão de Frequência (FDM, *Frequency Division Multiplexing*)**.

O canal físico (linha telefônica, 300–3400 Hz) é dividido em duas sub-bandas que abrigam simultaneamente os dois sinais transmitidos em sentidos opostos. Cada sinal ocupa uma faixa de frequência distinta e exclusiva, caracterizando a divisão do meio entre múltiplos fluxos – definição de multiplexação. Não há colisão entre os sinais porque operam em frequências diferentes.

Exercício 2

[1,0 ponto]

Dados do problema

O texto de 8 caracteres “Aprovado” é dividido em palavras de 2 bytes (16 bits). Usando a tabela ASCII, cada caractere é representado por um valor hexadecimal:

A	p	r	o	v	a	d	o
41	70	72	6F	76	61	64	6F

Agrupando em palavras de 16 bits (2 bytes por palavra):

Palavra	Hex	Binário (16 bits)
W_1 (A, p)	4170	0100 0001 0111 0000
W_2 (r, o)	726F	0111 0010 0110 1111
W_3 (v, a)	7661	0111 0110 0110 0001
W_4 (d, o)	646F	0110 0100 0110 1111

Cálculo no Emissor

O checksum é calculado pela soma em complemento de 1 de todas as palavras, seguida da inversão do resultado (complemento de 1 do total).

Passo 1 – Soma $W_1 + W_2$

$$\begin{array}{rcl}
 0100\ 0001\ 0111\ 0000 & (4170_{16} = 16752_{10}) \\
 +\ 0111\ 0010\ 0110\ 1111 & (726F_{16} = 29295_{10}) \\
 \hline
 1011\ 0011\ 1101\ 1111 & (B3DF_{16} = 46047_{10})
 \end{array}$$

Sem overflow (resultado $\leq$ FFFF).

Passo 2 – Soma com W_3

$$\begin{array}{rcl}
 1011\ 0011\ 1101\ 1111 & (B3DF_{16}) \\
 +\ 0111\ 0110\ 0110\ 0001 & (7661_{16} = 30305_{10}) \\
 \hline
 \text{carry:} & & \\
 1\ 0010\ 1010\ 0100\ 0000 & (\text{overflow!})
 \end{array}$$

Há overflow (bit 17 = 1). O carry é somado ao resultado (wrap-around):

$$0010\ 1010\ 0100\ 0000 + 1 = 0010\ 1010\ 0100\ 0001 = 2A41_{16}$$

Passo 3 – Soma com W_4

$$\begin{array}{rcl}
 0010\ 1010\ 0100\ 0001 & (2A41_{16}) \\
 +\ 0110\ 0100\ 0110\ 1111 & (646F_{16} = 25711_{10}) \\
 \hline
 1000\ 1110\ 1011\ 0000 & (8EB0_{16})
 \end{array}$$

Sem overflow.

Passo 4 – Complemento de 1 (inversão de todos os bits)

$$\text{Soma total} = 8\text{EB0}_{16} = 1000\ 1110\ 1011\ 0000_2$$

$$\text{Checksum} = \overline{8\text{EB0}}_{16} = 0111\ 0001\ 0100\ 1111_2 = 714\text{F}_{16}$$

O emissor transmite as quatro palavras seguidas do checksum:

$$\underbrace{W_1, W_2, W_3, W_4}_{\text{dados}}, \underbrace{714\text{F}_{16}}_{\text{checksum}}$$

Validação no Receptor

O receptor soma todas as palavras recebidas, **incluindo o checksum**. Se o resultado for FFFF_{16} (todos os bits em 1), a mensagem está íntegra.

Retomando: soma dos dados = 8EB0_{16}

$$\begin{array}{r} 1000\ 1110\ 1011\ 0000 \quad (8\text{EB0}_{16}) \\ +\ 0111\ 0001\ 0100\ 1111 \quad (714\text{F}_{16} - \text{checksum}) \\ \hline 1111\ 1111\ 1111\ 1111 \quad (\text{FFFF}_{16}) \end{array}$$

Resultado: A soma dá FFFF_{16} , confirmando que **não houve erros na transmissão**. O receptor aceita a mensagem como válida.

Exercício 3

[1,0 ponto]

Identificação do algoritmo

O algoritmo apresentado na Figura 2 é o **Go-Back-N com ARQ** (*Automatic Repeat reQuest*).

As evidências são:

- O emissor envia *múltiplos frames* sem aguardar o ACK de cada um individualmente (janela de transmissão > 1);
- Quando o Frame 2 é perdido, o receptor descarta os frames subsequentes que chegam fora de ordem (Frame 3 é ignorado pelo receptor);
- Após detectar a falha (ausência de ACK do Frame 2), o emissor *retransmite a partir do frame com erro* – comportamento típico do Go-Back-N.

Descrição passo-a-passo da transmissão

Passo 1. Envio do Frame 0: O emissor A transmite o Frame 0 ao receptor B.

Passo 2. ACK 1 (confirmação do Frame 0): O receptor B recebe o Frame 0 corretamente e envia o ACK 1, indicando que aguarda o próximo frame (Frame 1).

- Passo 3. Envio do Frame 1:** Ainda dentro da janela de transmissão, o emissor envia o Frame 1 sem aguardar o ACK anterior (transmissão em pipeline).
- Passo 4. ACK 2 (confirmação do Frame 1):** O receptor recebe o Frame 1 e envia ACK 2, aguardando agora o Frame 2.
- Passo 5. Envio do Frame 2 – perdido:** O emissor transmite o Frame 2, mas este é perdido no canal (indicado na figura como “Perdido”). O receptor nunca o recebe.
- Passo 6. Envio do Frame 3:** O emissor, sem saber da perda, continua e envia o Frame 3. O receptor B, porém, esperava o Frame 2 – ao receber o Frame 3 fora de ordem, o **descarta** (no Go-Back-N o receptor não armazena frames fora de ordem).
- Passo 7. Timeout / ausência de ACK 3:** O emissor percebe que não recebeu o ACK 3 dentro do tempo limite (timeout). Isso sinaliza que algo deu errado a partir do Frame 2.
- Passo 8. Retransmissão a partir do Frame 2:** O emissor **retransmite o Frame 2** (e retransmite também quaisquer frames enviados após ele, como o Frame 3, caso necessário – comportamento Go-Back-N).
- Passo 9. ACK 4 (confirmação do Frame 2 retransmitido e do Frame 3):** O receptor recebe o Frame 2 retransmitido corretamente e, em seguida, o Frame 3 (também retransmitido). Confirma com o **ACK 4**, indicando que ambos foram recebidos e que aguarda agora o Frame 4.

Exercício 4

[1,0 ponto]

Resposta

O TDM Estatístico (STDM) tende a ser **mais rápido** (maior eficiência de enlace) que o TDM Síncrono.

Justificativa

TDM Síncrono

No TDM Síncrono, o tempo do enlace é dividido em **slots fixos** e cada slot é atribuído a um canal de forma **determinística e periódica**, independentemente de o canal ter dados a enviar.

- Se um canal estiver **ocioso** (sem dados a transmitir), seu slot é desperdiçado – transmitido vazio.
- A capacidade total do enlace é distribuída igualmente entre todos os canais a todo momento.
- Exemplo: com 4 canais e 1 canal ocioso, 25% da banda é desperdiçada em cada quadro.

TDM Estatístico (Assíncrono)

No STDM, os slots **não são pré-atribuídos**. O multiplexador **aloca slots dinamicamente**, apenas para canais que possuem dados a transmitir.

- Canais ociosos **não consomem** slots; sua capacidade é redistribuída entre os canais ativos.
- Cada frame carrega um **cabeçalho** (overhead) identificando a qual canal cada slot pertence – isso é necessário porque a ordem não é mais fixa.
- A capacidade efetiva do enlace é aproveitada de forma **proporcional à demanda real** de cada canal.

Comparação resumida

Característica	TDM Síncrono	TDM Estatístico
Alocação de slots	Fixa	Dinâmica (por demanda)
Desperdício em canais ociosos	Sim	Não
Overhead de endereçamento	Não	Sim (pequeno)
Eficiência de enlace	Menor	Maior
Complexidade	Menor	Maior
Velocidade efetiva	Menor	Maior

Conclusão: O TDM Estatístico tende a ser mais rápido porque **elimina o desperdício de banda** causado por slots vazios. Ao alocar capacidade somente quando há tráfego real, a taxa de transferência útil sobre o mesmo enlace físico é maximizada. A desvantagem é a necessidade de overhead de identificação por frame, mas esse custo é geralmente superado pelo ganho de eficiência.

Exercício 5

[1,0 ponto]

Análise das alternativas

Regra: soma apenas as alternativas *verdadeiras*.

Alternativa (1) – VERDADEIRA

“Stop-and-wait com ARQ é um caso particular do Go-Back-N com ARQ, em que o tamanho da janela de transmissão é 1.”

Correto. No protocolo Go-Back-N, o emissor pode ter até W frames não confirmados em trânsito simultaneamente. Quando $W = 1$, o emissor envia um frame e aguarda o ACK antes de enviar o próximo – exatamente o comportamento do Stop-and-Wait. Portanto, Stop-and-Wait é de fato um caso especial de Go-Back-N com janela de tamanho 1.

Alternativa (2) – FALSA

“A largura de banda total mínima é de 540 kHz (5 canais $\times$ 100 kHz + 4 bandas de proteção $\times$ 10 kHz).”

Errado. O cálculo correto é:

$$BW_{total} = 5 \times 100 \text{ kHz} + 4 \times 10 \text{ kHz} = 500 + 40 = \mathbf{540 \text{ kHz}}$$

Aguarde – o valor de 540 kHz *está correto*. Entre 5 canais existem 4 bandas de proteção (entre canal 1–2, 2–3, 3–4 e 4–5). 540 kHz é o valor correto.

Revisando: a alternativa 2 é **VERDADEIRA**.

Alternativa (4) – VERDADEIRA

“Na transmissão serial síncrona existe overhead.”

Correto. Na transmissão síncrona, os dados são agrupados em *frames* que contêm campos de controle: delimitadores de início/fim de frame (flags), endereços, campos de controle de sequência e campos de verificação de erros (FCS/CRC). Esses campos adicionais constituem o *overhead* de protocolo – informação extra além dos dados úteis.

Alternativa (8) – FALSA

“Considerando a ordem *big-endian*, o valor em decimal que representa o byte 10110011 é 205.”

Errado. O byte 10110011 em binário, convertido para decimal é:

$$1 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0 = 128 + 32 + 16 + 2 + 1 = \mathbf{179}$$

O valor correto é **179**, não 205. A ordem *big-endian* se refere à ordenação de *bytes* na memória (byte mais significativo primeiro), não à conversão de um único byte de binário para decimal – portanto, não altera o resultado.

Alternativa (16) – FALSA

“Com números de sequência de 5 bits, os tamanhos máximos das janelas para *Stop-and-Wait* e *Selective Repeat* são, respectivamente, 31 e 16.”

Errado. Com $n = 5$ bits de número de sequência, temos $2^5 = 32$ números de sequência possíveis (0 a 31). Os tamanhos máximos de janela são:

Protocolo	Tamanho máximo de janela	Valor (n=5)
Stop-and-Wait	1	1
Go-Back-N	$2^n - 1$	31
Selective Repeat	2^{n-1}	16

A alternativa confunde Go-Back-N (janela = 31) com Stop-and-Wait. Para Stop-and-Wait, a janela é sempre **1** independentemente do número de bits de sequência.

Alternativa (32) – FALSA

“A distância de Hamming mínima no CRC equivale à quantidade de bits do código gerador menos um. Coloca-se essa distância mínima como preenchimento (padding) no cálculo do CRC.”

Errado em dois aspectos:

- A **distância de Hamming mínima** do CRC não é simplesmente “bits do gerador menos um” – ela depende do polinômio gerador específico escolhido e do tamanho da mensagem;
- O **padding** (preenchimento com zeros) no CRC corresponde ao *grau do polinômio gerador* (quantidade de bits do gerador menos 1), e é acrescentado ao *final da mensagem* antes da divisão, não relacionado à distância de Hamming. A distância de Hamming mínima é uma propriedade de detecção de erros do código, não um parâmetro de cálculo inserido na mensagem.

Resultado

As alternativas verdadeiras são: (1), (2) e (4).

$$\boxed{\text{Soma} = 1 + 2 + 4 = 7}$$

Exercício 6

[1,0 ponto]

Alternativa correta

b) unipolar, RZ, NRZ, polar

Análise de cada sinal

Sinal (a) – dados: 1 0 1 1 0

O sinal apresenta:

- Para o bit **1**: nível de tensão **positivo** durante todo o período do bit (sem retorno a zero).
- Para o bit **0**: nível **zero** durante todo o período do bit.

Usa apenas dois níveis: positivo e zero (nunca negativo). $\Rightarrow$ **Unipolar NRZ** (Non-Return-to-Zero unipolar).

Sinal (b) – dados: 0 1 0 0 1

O sinal apresenta:

- Para o bit **1**: o sinal vai ao nível positivo na **primeira metade** do período e **retorna a zero** na segunda metade.
- Para o bit **0**: permanece em zero.

Há retorno explícito ao nível zero dentro do período de cada bit. $\Rightarrow$ **RZ** (Return-to-Zero).

Sinal (c) – dados: 0 1 0 0 1

O sinal apresenta:

- Para o bit **1**: nível positivo durante **todo** o período.
- Para o bit **0**: nível negativo (ou zero) durante todo o período, **sem** retorno a zero no meio do bit.

Usa dois níveis fixos sem retorno ao zero: $\Rightarrow$ **NRZ**.

Sinal (d) – dados: 0 1 0 0 1

O sinal apresenta:

- Usa **dois níveis de tensão** (positivo e negativo), porém com uma transição **no meio** do período de cada bit.
- Para o bit **1**: transição de positivo para negativo (ou vice-versa) no centro do bit.
- Para o bit **0**: sem transição no centro.

Característica de codificação **Polar** com transição diferencial. $\Rightarrow$ **Polar** (ex.: Manchester diferencial ou NRZ-I).

Resposta final: alternativa b) unipolar | RZ | NRZ | polar

Exercício 7

[0,4 ponto]

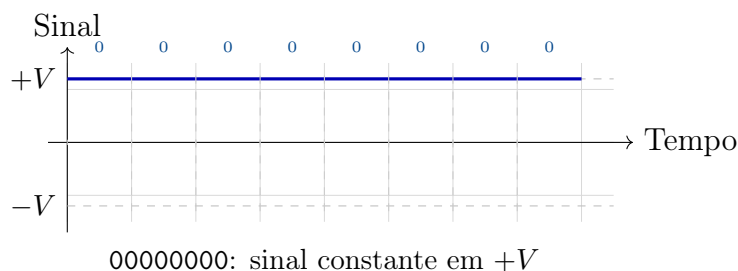
Método NRZ-L (*Non-Return-to-Zero Level*):

- Bit **0** $\rightarrow$ nível de sinal **positivo** ($+V$)
- Bit **1** $\rightarrow$ nível de sinal **negativo** ($-V$)

O enunciado informa que o **último nível de sinal anterior foi positivo**.

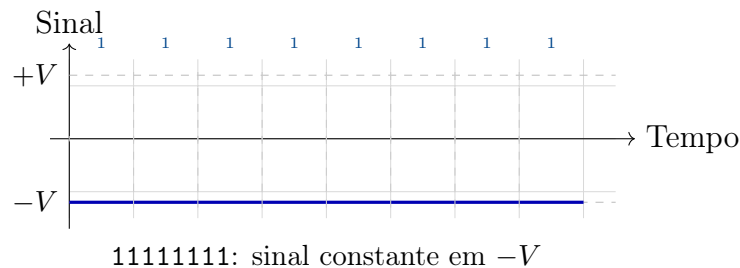
a) Fluxo: 00000000

Todos os bits são 0, portanto o sinal permanece em nível positivo ($+V$) durante toda a sequência. Como já estava em $+V$ antes do início, não há transição.

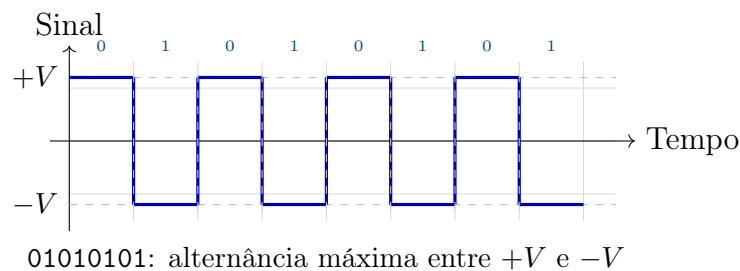
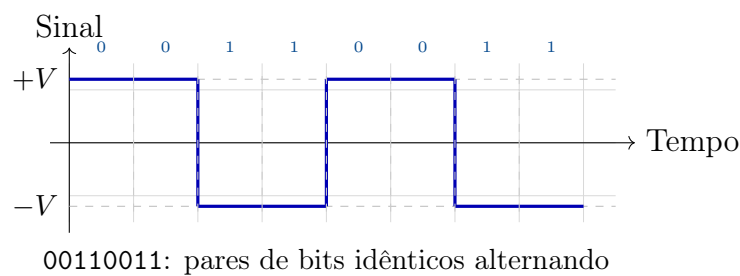


b) Fluxo: 11111111

Todos os bits são 1, portanto o sinal permanece em nível negativo ($-V$) durante toda a sequência. Há uma transição de $+V$ para $-V$ no instante inicial (antes do primeiro bit).

**c) Fluxo: 01010101**

O sinal alterna entre $+V$ (bit 0) e $-V$ (bit 1) a cada bit. O sinal inicial começa em $+V$ (bit 0).

**d) Fluxo: 00110011****Exercício 8**

[0,6 ponto]

Regra Manchester (IEEE 802.3)

- Bit 0 → transição de $+V$ para $-V$ no meio do período (descida).
- Bit 1 → transição de $-V$ para $+V$ no meio do período (subida).

- As transições no meio do bit são **obrigatórias** e servem também como sincronismo.
- Podem ocorrer transições adicionais entre bits consecutivos de mesmo tipo para garantir a transição obrigatória no centro.

a) Sequência: 00000000

Todos os bits são 0 $\Rightarrow$ cada bit: $+V$ na primeira metade $\rightarrow -V$ na segunda metade. Entre bits consecutivos iguais (0 $\rightarrow$ 0), há transição de retorno ($-V \rightarrow +V$) na borda entre os bits.

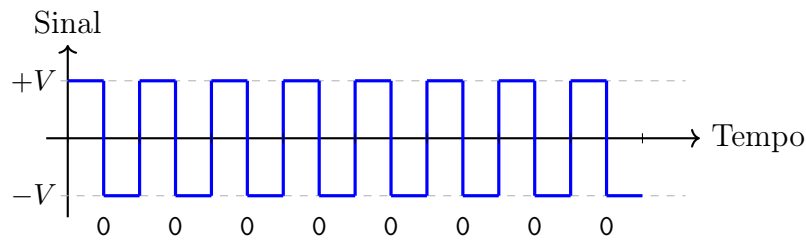


Figura 1: Manchester – sequência 00000000: transições obrigatórias no centro de cada bit (0 = descida)

b) Sequência: 11111111

Todos os bits são 1 $\Rightarrow$ cada bit: $-V$ na primeira metade $\rightarrow +V$ na segunda metade. Entre bits consecutivos iguais (1 $\rightarrow$ 1), há transição de retorno ($+V \rightarrow -V$) na borda entre os bits.

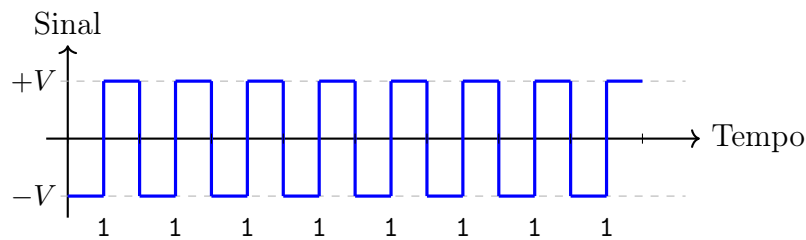


Figura 2: Manchester – sequência 11111111: transições obrigatórias no centro de cada bit (1 = subida)

c) Sequência: 01010101

Bits alternados: entre bits diferentes não há transição na borda (a segunda metade de um bit já está no nível correto para a primeira metade do próximo).

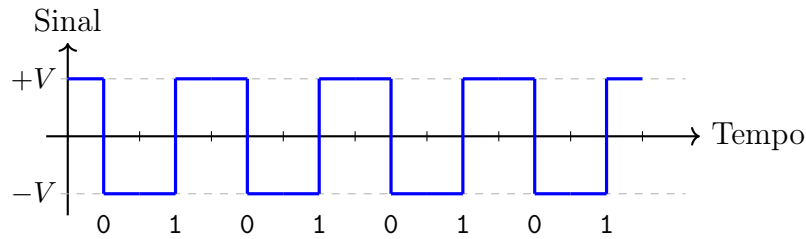


Figura 3: Manchester – sequência 01010101: sem transições extras nas bordas (bits alternados)

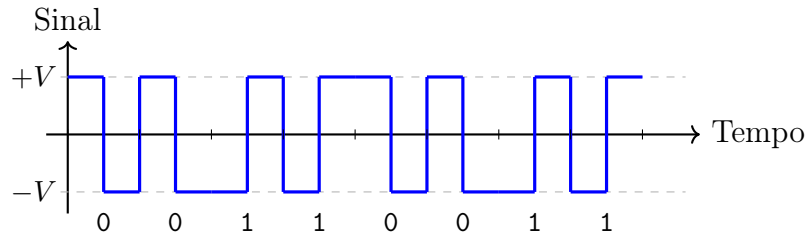


Figura 4: Manchester – sequência 00110011

d) Sequência: 00110011

Exercício 9

Identificação da modulação

A modulação realizada na linha 24 da função `modulacao` do arquivo `multiplexador.py` é conhecida como **AM DSB-SC** (*Amplitude Modulation – Double Side Band, Suppressed Carrier*). Conceitualmente, esse processo transla um sinal de baixa frequência (banda base, como o áudio) para uma região de frequência mais alta, permitindo que múltiplos sinais sejam transmitidos no mesmo meio sem sobreposição. Essa é a base do FDM (*Frequency Division Multiplexing*).

Representação matemática

Matematicamente, a modulação ocorre pela multiplicação direta do sinal de mensagem $m(t)$ por uma onda portadora $c(t) = \cos(2\pi f_c t)$ no domínio do tempo:

$$s(t) = m(t) \cdot \cos(2\pi f_c t) \quad (1)$$

Pelo teorema da modulação, também chamado de propriedade de deslocamento da Transformada de Fourier, essa multiplicação no domínio do tempo resulta na convolução dos espectros no domínio da frequência. Como a transformada de um cosseno consiste em dois impulsos, localizados em $+f_c$ e $-f_c$, o espectro da mensagem $M(f)$ é deslocado para a frequência da portadora, gerando duas bandas laterais:

$$S(f) = \frac{1}{2} [M(f - f_c) + M(f + f_c)] \quad (2)$$

Justificativa do gráfico

Como os sinais gerados originalmente na função `geraSinais()` são ondas senoidais puras, com frequências de 440 Hz, 880 Hz e 1760 Hz, seus espectros $M(f)$ originais são compostos apenas por picos isolados nessas frequências.

Ao modular, por exemplo, o sinal A (440 Hz) com a portadora A (4000 Hz), as novas frequências geradas serão:

$$4000 - 440 = 3560 \text{ Hz} \quad \text{e} \quad 4000 + 440 = 4440 \text{ Hz}$$

O mesmo cálculo se aplica aos demais canais:

- **Canal B:** o sinal de 880 Hz modulado pela portadora de 10000 Hz resulta nos picos em 9120 Hz e 10880 Hz.
- **Canal C:** o sinal de 1760 Hz modulado pela portadora de 14000 Hz resulta nos picos em 12240 Hz e 15760 Hz.

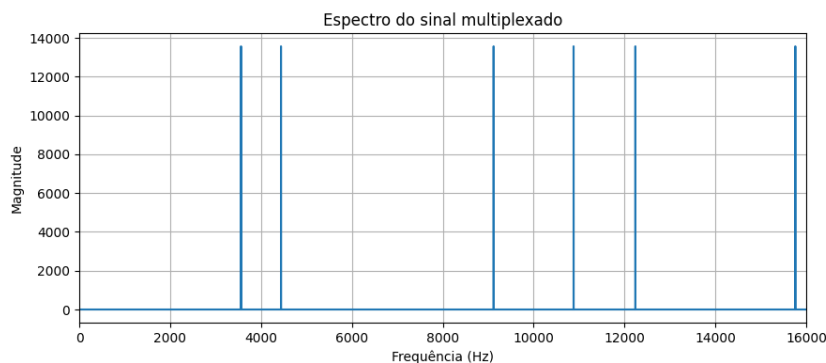


Figura 5: Espectro do sinal multiplexado mostrando a modulação AM DSB-SC.

Ao juntarmos os três canais, justificam-se os seis picos isolados observados no espectro multiplexado da Figura 5.

Exercício 10

Interpretação dos espectrogramas

As figuras geradas pela função `compara_espectro()` representam **espectrogramas**, isto é, representações tridimensionais do espectro achatadas em duas dimensões. Nessa visualização:

- O eixo horizontal (X) representa o **tempo**, em segundos.
- O eixo vertical (Y) representa a **frequência**, em Hz.
- A cor, em forma de mapa de calor, indica a **amplitude** ou potência do sinal naquela frequência em determinado instante. Tons mais claros, como amarelo e verde, indicam maior energia.

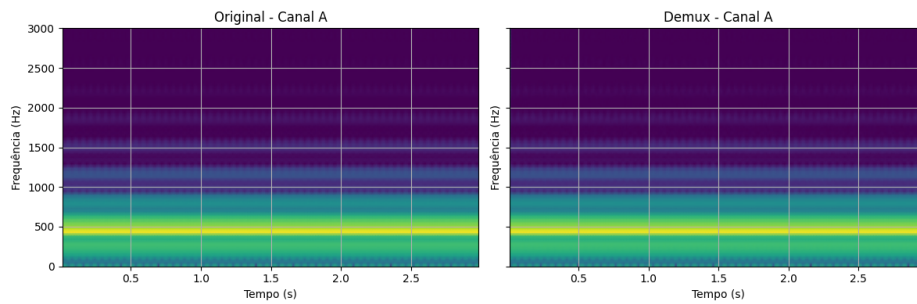


Figura 6: Comparação de espectrogramas original e demultiplexado do Canal A.

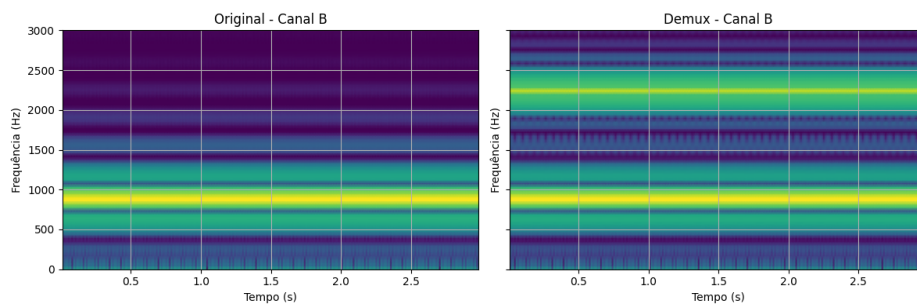


Figura 7: Comparação de espectrogramas original e demultiplexado do Canal B.

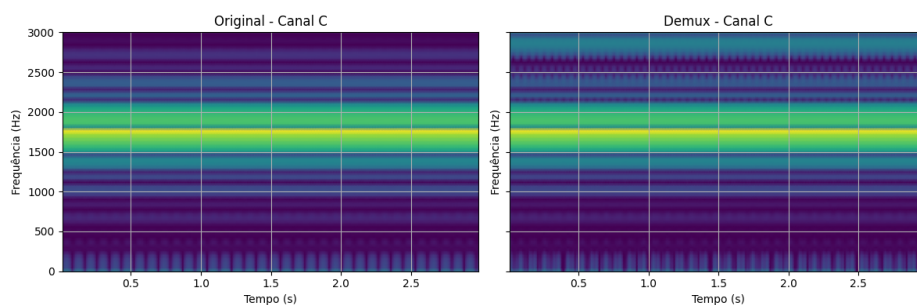


Figura 8: Comparação de espectrogramas original e demultiplexado do Canal C.

Análise dos resultados

Análise do original: nos gráficos à esquerda, correspondentes ao sinal original, como os sinais de teste eram ondas senoidais puras, o espectrograma revela uma única linha horizontal luminosa constante durante todo o tempo de gravação. Isso indica que a energia do sinal está concentrada nas frequências de 440 Hz, 880 Hz e 1760 Hz.

Análise do demultiplexado: nos gráficos à direita, correspondentes ao sinal demultiplexado, observa-se que a linha horizontal principal **reaparece na mesma frequência** correspondente ao sinal original. Isso comprova a eficácia do processo implementado: os filtros passa-banda conseguiram isolar corretamente cada canal na região de alta frequência, a demodulação transladou o sinal de volta para a banda base e o filtro passa-baixa eliminou as frequências indesejadas, resultando na **recuperação bem-sucedida do sinal da mistura multiplexada**.

Avaliação da eficiência: a recuperação do tom fundamental é altamente eficiente. Contudo, é possível observar pequenos “fantasmas” ou ruídos residuais, representados por faixas com coloração levemente diferente do fundo azul escuro. Esses artefatos são causados por imperfeições inerentes aos filtros digitais utilizados. Os filtros *Butterworth* empregados têm ordem fixa igual a 6, o que gera uma atenuação gradual (*roll-off*) em vez de um corte abrupto e ideal. Assim, pequenos fragmentos de outras frequências, ou resíduos de alta frequência ($2f_c$) gerados na demodulação, podem passar pelo filtro e criar esses sutis artefatos visuais. Mesmo assim, o sistema cumpre satisfatoriamente seu papel.

Exercício 11 (Bônus)

Implementação proposta

Para a resolução deste bônus, os scripts originais foram unificados e refatorados em uma única classe `SistemaComunicacao`, baseada na biblioteca `librosa`. O processo foi estruturado para atender aos subitens solicitados.

a) Espectro dos sinais modulados

O código utiliza `scipy.fft` para gerar os gráficos no domínio da frequência para as três bandas após a modulação AM DSB-SC, com portadoras em 4000 Hz, 10000 Hz e 14000 Hz.

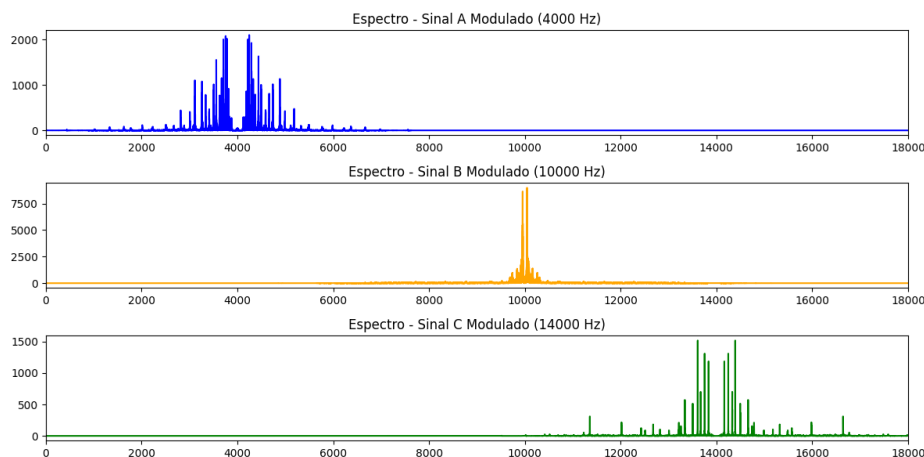


Figura 9: Espectro dos sinais modulados individualmente.

b) Espectro multiplexado

Após somar os três sinais modulados, o espectro FDM consolidado foi plotado, exibindo as faixas ocupadas pelos três instrumentos simultaneamente.

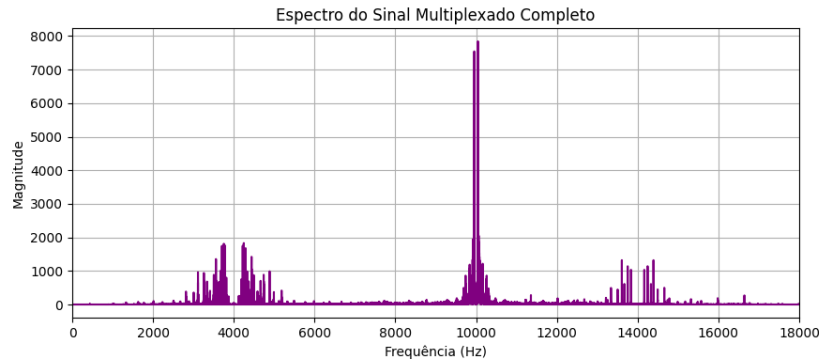


Figura 10: Espectro do sinal multiplexado completo.

c) Salvamento do sinal multiplexado

O sinal combinado foi salvo localmente usando a biblioteca `soundfile`, no arquivo `muxed_audio.wav`.

d) e e) Demultiplexação e filtragem

O processo inverso aplica três filtros Passa-Banda do tipo *Butterworth*, de 6ª ordem, nas portadoras alvo para isolar os canais. Em seguida, os sinais são multiplicados novamente por suas respectivas portadoras e passam por filtros Passa-Baixa com frequência de corte em 3000 Hz, retornando à banda base.

f) Comparação dos áudios

Ao ouvir e comparar os arquivos `base_[Canal].wav`, correspondentes ao sinal ajustado antes do FDM, com os arquivos `demux_channel_[Canal].wav`, notou-se que as características acústicas da voz ou da música foram mantidas inteligíveis e no tom correto.

A limitação a um espectro estreito de 3000 Hz, necessária para evitar *overlapping* entre os canais, atenuou os agudos e conferiu uma característica semelhante a “som telefônico”. Apesar disso, essa limitação garantiu a separação dos canais sem interferência cruzada significativa.

g) Código e áudios anexos

O script final, os arquivos de origem de teste e os resultados em `.wav` acompanham os anexos desta submissão.

Código-fonte refatorado

```
1 import numpy as np
2 import soundfile as sf
3 import matplotlib.pyplot as plt
4 from scipy.fft import fft, fftfreq
5 from scipy.signal import butter, lfilter
6 import librosa
```

```
7 import os
8
9 class SistemaComunicacao:
10     def __init__(self):
11         # Taxa de amostragem padronizada para todos os audios
12         self.fs = 44100
13         # Frequências das portadoras escolhidas
14         self.fc_A = 4000    # Canal A (ex: piano.wav)
15         self.fc_B = 10000   # Canal B (ex: bateria.wav)
16         self.fc_C = 14000   # Canal C (ex: violao.wav)
17
18         # Filtro de corte da banda base. Limitado a 3kHz para
19         evitar
20         # sobreposicao (overlapping) na multiplexacao FDM.
21         self.cutoff_baseband = 3000
22         self.order = 6 # Ordem dos filtros Butterworth
23
24     # === Funcoes Auxiliares de Filtros ===
25     def lowpass_filter(self, signal, cutoff, fs, order):
26         nyq = 0.5 * fs
27         b, a = butter(order, cutoff/nyq, btype='low')
28         return lfilter(b, a, signal)
29
30     def bandpass_filter(self, signal, lowcut, highcut, fs, order):
31         nyq = 0.5 * fs
32         b, a = butter(order, [lowcut/nyq, highcut/nyq], btype='band')
33         return lfilter(b, a, signal)
34
35     def normalize(self, signal):
36         return signal / np.max(np.abs(signal))
37
38     # === Etapa 1: Preparacao do Sinal ===
39     def carrega_audios(self):
40         print("Carregando arquivos de audio via librosa...")
41         a, _ = librosa.load("piano.wav", sr=self.fs, mono=True)
42         b, _ = librosa.load("bateria.wav", sr=self.fs, mono=True)
43         c, _ = librosa.load("violao.wav", sr=self.fs, mono=True)
44
45         # Truncar todos os audios para o menor tamanho
46         encontrado para podermos somar
47         min_len = min(len(a), len(b), len(c))
48         a, b, c = a[:min_len], b[:min_len], c[:min_len]
49
50         # Pre-filtragem anti-aliasing (Passa-Baixa em 3000 Hz)
51         a = self.lowpass_filter(a, self.cutoff_baseband, self.fs, self.order)
52         b = self.lowpass_filter(b, self.cutoff_baseband, self.fs, self.order)
```



```
51         c = self.lowpass_filter(c, self.cutoff_baseband, self.fs
52                                   , self.order)
53
54         # Salva os arquivos base normalizados
55         sf.write("base_A.wav", self.normalize(a), self.fs)
56         sf.write("base_B.wav", self.normalize(b), self.fs)
57         sf.write("base_C.wav", self.normalize(c), self.fs)
58
59         return a, b, c, min_len
60
61     # === Etapa 2: Modulacao e Multiplexacao ===
62     def multiplexacao(self):
63         a, b, c, length = self.carrega_audios()
64         t = np.arange(length) / self.fs
65
66         # Modulacao individual de cada sinal (AM DSB-SC)
67         a_mod = a * np.cos(2 * np.pi * self.fc_A * t)
68         b_mod = b * np.cos(2 * np.pi * self.fc_B * t)
69         c_mod = c * np.cos(2 * np.pi * self.fc_C * t)
70
71         # Exibe os espectros individuais
72         self.plota_espectros_individuais(a_mod, b_mod, c_mod,
73                                           length)
74
75         # Soma e normaliza
76         muxed = self.normalize(a_mod + b_mod + c_mod)
77         sf.write("muxed_audio.wav", muxed, self.fs)
78
79         # Exibe o espectro multiplexado
80         self.plota_espectro_multiplexado(muxed, length)
81
82         return muxed, length
83
84     # === Etapa 3: Demultiplexacao e Recuperacao ===
85     def demultiplexacao(self, muxed, length):
86         t = np.arange(length) / self.fs
87         carriers = {'A': self.fc_A, 'B': self.fc_B, 'C': self.
88                     fc_C}
89
90         print("Iniciando Demultiplexacao...")
91         for label, fc in carriers.items():
92             band = self.bandpass_filter(muxed, fc - self.
93                                         cutoff_baseband, fc + self.cutoff_baseband, self.
94                                         fs, self.order)
95             demod = band * np.cos(2 * np.pi * fc * t)
96             baseband = self.lowpass_filter(demod, self.
97                                             cutoff_baseband, self.fs, self.order)
98
99             baseband_norm = self.normalize(baseband)
100            sf.write(f"demux_channel_{label}.wav", baseband_norm
101                    , self.fs)
```

```
95
96     # === Funcoes Graficas omitidas no relatorio por
97         simplicidade ===
98     # (Elas utilizam scipy.fft para exibir as frequencias na
99         tela)
100
101     def plota_espectros_individuais(self, a, b, c, l): pass
102     def plota_espectro_multiplexado(self, muxed, l): pass
103
104 if __name__ == "__main__":
105     sistema = SistemaComunicacao()
106     sinal_multiplexado, min_length = sistema.multiplexacao()
107     sistema.demultiplexacao(sinal_multiplexado, min_length)
```